Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе № 1
Тема: «Знакомство с Linux/Unix и средой программирования. POSIX-совместимая файловая система»

Выполнил:
ст. гр. 450502 Новицкий Е.Ю

Проверил:
Благиных В. Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить POSIX-совместимую файловую систему, команды и утилиты оболочки.

Разработать программу dirwalk, сканирующую файловую систему и выводящую в stdout информацию в соответствии с опциями программы.

Формат вывода аналогичен формату вывода утилиты find. dirwalk [dir] [options] – dir начальный каталог.

Опции options:

-l – только символические ссылки (-type l)

-d – только каталоги (-type d)

-f – только файлы (-type f)

-s – сортировка по LC_COLLOCATE (-type f)

Опции могут быть указаны как перед каталогом, так и после. Опции могут быть указаны как раздельно, так и вместе (-l -d, -ld). Если опции ldf опущены, выводятся каталоги, файлы и ссылки.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа dirwalk выполняет рекурсивный обход файловой системы, начиная с заданного каталога (по умолчанию ./), и выводит пути объектов в stdout в формате, аналогичном find (один путь на строку).

Алгоритм работы:

1) Разбор аргументов командной строки:
   - поддерживаются опции -l, -d, -f, -s;
   - опции можно указывать до и после каталога, а также объединять (например, -ld);
   - если фильтры -l/-d/-f не заданы, выводятся каталоги, регулярные файлы и симлинки.

2) При наличии -s включается локаль LC_COLLATE для корректной сортировки строк.

3) Выполняется рекурсивный обход дерева каталогов:
   - для каждого пути вызывается lstat();
   - тип объекта определяется через S_ISLNK/S_ISDIR/S_ISREG;
   - если объект подходит под фильтр, путь либо печатается сразу, либо сохраняется в динамический массив (режим -s).

4) Для режима -s после завершения обхода список путей сортируется по strcoll() и выводится.

5) Освобождается вся выделенная память, программа завершается с кодом результата.

Программа реализована на переносимых POSIX-вызовах (opendir/readdir/closedir/lstat/getopt), без Linux-специфичных расширений.

2.2 Описание основных функций и модулей

2.2.1 Модуль list (include/list.h, src/list.c)

Структура list:
- char **data – массив строк (путей),
- size_t lenght – текущее число элементов,
- size_t capacity – выделенная емкость.

Функции:
- init() – инициализация пустого массива.
- add(list* ls, char* str) – добавление пути:
  при нехватке места емкость увеличивается.
- list_free() – освобождение памяти всех строк и массива.
- print_list(list* ls) – вывод содержимоого списка на экран.

2.2.2 Модуль walker (include/dirwalk.h, src/dirwalk.c)

Основная функция:
- walk (const char *start_dir, _Bool include_files, _Bool include_dirs, _Bool include_links)
  Выполняет обход дерева от start_dir.
- sort(list* ls) – выполняет сортировку списка в соответствии с LC_COLLATE.

2.2.3 Главный модуль (src/main.c)

Функция main():

1) обрабатывает аргументы;
2) запускает walk;
3) при -s включает setlocale(LC_COLLATE, "");
4) при -s сортирует и выводит накопленные пути;
5) освобождает память и возвращает код завершения.

2.3 Функциональная структура проекта

Проект разделен на интерфейсы (.h) и реализации (.c):
- include/list.h: интерфейс динамического массива строк;
- include/dirwalk.h: интерфейс обхода файловой системы;
- src/list.c: реализация хранения/сортировки путей;
- src/dirwalk.c: реализация рекурсивного обхода;
- src/main.c: точка входа и связывание модулей;
- Makefile: сборка исполняемого файла dirwalk.

Такое разбиение упрощает сопровождение: логика опций, обхода и хранения данных изолирована по модулям.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Сборка выполняется с помощью Makefile.
Команды:
1) Полная пересборка: make clean && make;
2) Обычная сборка: make;
3) Очистка артефактов: make clean.
Результат сборки: исполняемый файл ./dirwalk.

Файл Makefile

```
CC = gcc

CFLAGS = -std=c11 -pedantic -Wall -Wextra

LDFLAGS =

MODE ?= debug

ifeq ($(MODE),debug)

   CFLAGS += -g -ggdb

else ifeq ($(MODE),release)

   CFLAGS += -Werror

else

   $(error Unknown MODE: $(MODE))

endif

SRCS = main.c walk.c list.c

OBJS = $(SRCS:.c=.o)

TARGET = dirwalk

all: $(TARGET)

$(TARGET): $(OBJS)

        $(CC) $(LDFLAGS) -o $@ $^

%.o: %.c

        $(CC) $(CFLAGS) -c $< -o $@

clean:

        rm -f $(OBJS) $(TARGET)
```

.PHONY: all clean

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

Синтаксис:
dirwalk [dir] [options]
Опции:
-l только симлинки;
-d только каталоги;
-f только регулярные файлы;
-s сортировка по LC_COLLATE.

Проверка корректности (сравнение с find + wc). Попарные значения должны совпасть.

```
dirwalk . -fdl | wc -l
Cannot open directory './cups/certs': Permission denied
   267
dirwalk . -d | wc -l
Cannot open directory './cups/certs': Permission denied
     33
dirwalk . -f | wc -l
Cannot open directory './cups/certs': Permission denied
    231
dirwalk . -l | wc -l
Cannot open directory './cups/certs': Permission denied
      3
find /private/etc \( -type f -o -type d -o -type l \) | wc -l
find: /private/etc/cups/certs: Permission denied
    267
find /private/etc -type d | wc -l
find: /private/etc/cups/certs: Permission denied
     33
find /private/etc -type f | wc -l
find: /private/etc/cups/certs: Permission denied
    231
find /private/etc -type l | wc -l
find: /private/etc/cups/certs: Permission denied
      3
```

Примечание для macOS:
В большинстве систем /etc является симлинком на /private/etc, поэтому для демонстрации всех трех типов файлов корректнее использовать именно /private/etc.

Вывод

При выполнении лабораторной работы была изучена работа с файлами в операционной системе Linux.

В рамках выполнения работы была создана программа, которая анализирует файлы и выводит информацию в соответствии с заданными параметрами. Программа поддерживает различные опции, позволяющие гибко настраивать процесс сканирования и формат вывода данных.

ПРИЛОЖЕНИЕ А
(Программный код)

Файл list.c

```
#include "../include/list.h"

#define INITIAL_CAPACITY 8

list* init(void) {
list *ls = (list*)malloc(sizeof(list));
if (!ls) return NULL;
ls->length = 0;
ls->capacity = INITIAL_CAPACITY;
ls->data = (char**)malloc(sizeof(char*) * ls->capacity);
if (!ls->data) {
free(ls);
return NULL;
}
return ls;
}

void list_free(list* ls) {
if (!ls) return;
for (size_t i = 0; i < ls->length; ++i) {
free(ls->data[i]);
}
free(ls->data);
ls->data = NULL;
ls->length = 0;
ls->capacity = 0;
free(ls);
}

void add(list* ls, char* str) {
if (!ls) return;
if (ls->length >= ls->capacity) {
resize(ls);
}
ls->data[ls->length++] = str;
}

void resize(list* ls) {
size_t new_capacity = ls->capacity ? (ls->capacity < 128 ? (size_t)(ls->capacity * 1.5) : ls->capacity +
10)
: INITIAL_CAPACITY;
char** new_data = (char**)realloc(ls->data, sizeof(char*) * new_capacity);
if (!new_data) {
return;
}
ls->data = new_data;
ls->capacity = new_capacity;
}

int merge(list* dst, list* src) {
size_t required;
```

```
size_t new_capacity;
char** new_data;

required = dst->length + src->length;
if (required <= dst->capacity) {
memcpy(dst->data + dst->length, src->data, sizeof(char*) * src->length);
dst->length = required;
src->length = 0;
return 0;
}

new_capacity = dst->capacity ? dst->capacity : INITIAL_CAPACITY;
while (new_capacity < required) {
if (new_capacity > SIZE_MAX / 2) {
new_capacity = required;
break;
}
new_capacity *= 2;
}

if (new_capacity > SIZE_MAX / sizeof(char*)) return -1;
new_data = (char**)realloc(dst->data, sizeof(char*) * new_capacity);
if (!new_data) return -1;

dst->data = new_data;
dst->capacity = new_capacity;
memcpy(dst->data + dst->length, src->data, sizeof(char*) * src->length);
dst->length = required;
src->length = 0;
return 0;
}

void print_list(list *ls) {
if (!ls) return;
for (size_t i = 0; i < ls->length; ++i) {
printf("%s\n", ls->data[i]);
}
}
```

Файл dirwalk.c

```
#ifndef _DEFAULT_SOURCE
#define _DEFAULT_SOURCE
#include <errno.h>
#include <string.h>
#endif
#include "../include/dirwalk.h"


list* walk(char* start_dir, _Bool include_files, _Bool include_dirs, _Bool include_links) {
DIR* dir = opendir(start_dir);
list *res = init();

if (include_dirs) {
size_t len = strlen(start_dir) + 1;
char* owned_path = (char*)malloc(len);
memcpy(owned_path, start_dir, len);
```

```
add(res, owned_path);
}

if (!dir) {
fprintf(stderr, "%s: %s\n", start_dir, strerror(errno));
return res;
}

struct dirent* entry;
struct stat st;
while ((entry = readdir(dir)) != NULL) {
if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0) {
continue;
}

char fullpath[1024];
snprintf(fullpath, sizeof(fullpath), "%s/%s", start_dir, entry->d_name);
if (lstat(fullpath, &st) != 0) {
continue;
}

if (S_ISREG(st.st_mode) && include_files || S_ISLNK(st.st_mode) && include_links) {
size_t len = strlen(fullpath) + 1;
char* owned_path = (char*)malloc(len);
memcpy(owned_path, fullpath, len);
add(res, owned_path);
}

if (S_ISDIR(st.st_mode)) {
list* child = walk(fullpath, include_files, include_dirs, include_links);
if (child) {
merge(res, child);
list_free(child);
}
}
}

closedir(dir);
return res;
}

list* sort(list* ls) {
qsort(ls->data, ls->length, sizeof(char*), cmp_locale);
return ls;
}

int cmp_locale(const void *a, const void *b) {
const char* const* left = (const char* const*)a;
const char* const* right = (const char* const*)b;
return strcoll(*left, *right);
}
```

Файл main.c

```
#include <locale.h>
#include <unistd.h>
#include <getopt.h>
```

```
#include <locale.h>
#include "../include/dirwalk.h"
#include "../include/list.h"

int main(int argc, char* argv[]) {
_Bool include_dirs = 0, include_files = 0, include_links = 0, with_sort = 0;
char* start_dir = ".";
list *res;

if (argc >= 2) {
if (argv[1][0] != '-') {
start_dir = argv[1];
optind = 2;
}

int opt;
while ((opt = getopt(argc, argv, "lfds")) != -1) {
switch (opt) {
case 'l': include_links = 1; break;
case 'f': include_files = 1; break;
case 'd': include_dirs = 1; break;
case 's': with_sort = 1; break;
case '?': return 1;
}
}
}

if (!(include_dirs || include_files || include_links))
res = walk(start_dir, 1, 1, 1);
else
res = walk(start_dir, include_files, include_dirs, include_links);

if (!res)
return 1;
if (with_sort) {
setlocale(LC_COLLATE, "");
sort(res);
}

print_list(res);
list_free(res);
return 0;
}
```